Lecture 6: Memory Hierarchy - Cache Memory

1. The Memory Hierarchy Concept (Registers, Cache, Main Memory, Secondary Storage)

Modern computer systems use a **memory hierarchy** to balance the conflicting requirements of speed, capacity, and cost. The idea is to provide fast access to frequently used data by keeping it in smaller, faster, and more expensive memory technologies, while larger, slower, and cheaper technologies store less frequently accessed data.

The hierarchy typically looks like this, from fastest/smallest/most expensive to slowest/largest/cheapest:

1. **Registers:**
    - **Location:** Inside the CPU.
    - **Speed:** Extremely fast (single clock cycle access).
    - **Capacity:** Very small (tens to hundreds of bytes).
    - **Cost:** Highest per bit.
    - **Purpose:** Hold data currently being processed by the CPU, temporary operands, addresses, etc.
2. **Cache Memory (L1, L2, L3):**
    - **Location:** Close to the CPU, often on the same chip.
    - **Speed:** Very fast (tens of clock cycles).
    - **Capacity:** Small (kilobytes to tens of megabytes).
    - **Cost:** High per bit, but less than registers.
    - **Purpose:** Store copies of data and instructions from main memory that are likely to be accessed soon. Acts as a buffer between CPU and main memory.
    - **Levels:** Often multi-level (L1 - fastest, smallest, per-core; L2 - larger, slower, per-core or shared; L3 - largest, slowest, shared across all cores).
3. **Main Memory (RAM - Random Access Memory):**
    - **Location:** Separate modules on the motherboard.
    - **Speed:** Relatively fast (hundreds of clock cycles, nanoseconds).
    - **Capacity:** Medium (gigabytes).
    - **Cost:** Medium per bit.
    - **Purpose:** Stores currently running programs and their data. It's volatile (contents lost when power is off).

4. **Secondary Storage (Mass Storage):**
    - **Location:** Hard drives (HDDs), Solid State Drives (SSDs), USB drives, network storage.
    - **Speed:** Slow (milliseconds, orders of magnitude slower than RAM).
    - **Capacity:** Large (terabytes to petabytes).
    - **Cost:** Lowest per bit.
    - **Purpose:** Permanent storage of programs, operating system, and user data. Non-volatile (contents persist when power is off).

Why Hierarchy Works: The effectiveness of the memory hierarchy relies heavily on the principle of **Locality of Reference**.

2. Locality of Reference (Temporal and Spatial)

Locality of reference is the tendency of a processor to access the same set of memory locations repetitively over a short period of time. This fundamental principle makes caching effective.

- **Temporal Locality:**
    - If a data item is accessed now, it is likely to be accessed again in the near future.
    - **Example:** Variables in a loop, function parameters, frequently used instructions.
    - **Benefit for Cache:** If an item is in the cache, it can be quickly re-accessed from the cache when needed again.
- **Spatial Locality:**
    - If a program accesses a particular memory location, it is likely to access nearby memory locations soon.
    - **Example:** Accessing elements of an array sequentially, fetching instructions sequentially from memory.
    - **Benefit for Cache:** When a block of data is fetched from a slower level of memory (e.g., main memory) into a faster level (e.g., cache), it brings not only the requested data but also surrounding data that is likely to be needed soon. This larger chunk is called a **cache line** or **cache block**.

3. Cache Memory

Cache memory is a small, very fast memory that stores copies of data from frequently used main memory locations. Its goal is to reduce the average time to access data from main memory.

3.1. Cache Principles (Hit, Miss)

- **Cache Hit:** Occurs when the CPU requests data, and that data is found in the cache. This is very fast.
- **Cache Miss:** Occurs when the CPU requests data, and that data is *not* found in the cache.
    - When a miss occurs, the data must be fetched from the next lower (slower) level of the memory hierarchy (e.g., main memory).
    - The requested data, along with a block of surrounding data (the cache line), is loaded into the cache, replacing an existing cache line if necessary.
    - This is a slower operation.
- **Hit Rate / Hit Ratio:** The percentage of memory accesses that result in a cache hit. A higher hit rate means better performance.
- **Miss Rate:** The percentage of memory accesses that result in a cache miss (1−Hit Rate).
- **Average Memory Access Time (AMAT):** A key performance metric for memory systems.
    - AMAT = (Hit Rate × Cache Access Time) + (Miss Rate × Miss Penalty)
    - **Miss Penalty:** The time taken to fetch data from the next lower level of memory and load it into the cache after a miss.

3.2. Cache Mapping Techniques

When a block of data is brought from main memory into the cache, where exactly in the cache does it go? Mapping techniques determine this placement.

1. **Direct Mapped Cache:**
    - **Simplest:** Each block from main memory has only *one* specific location where it can be placed in the cache.
    - **Mechanism:** The main memory address is divided into three parts: **Tag, Index, Block Offset.**
        - **Index:** Determines which "line" or "slot" in the cache the block maps to.
        - **Tag:** Used to identify *which* specific block from main memory (since many main memory blocks can map to the same cache line index) is currently stored in that cache line.
        - **Block Offset:** Specifies the position of the desired byte within the cache line.

- **Advantages:** Simple hardware, fast lookup.
- **Disadvantages:** High potential for **conflict misses** if frequently accessed blocks map to the same cache line, even if other cache lines are empty.

2. **Fully Associative Cache:**
    - **Most Flexible:** Any block from main memory can be placed in *any* available location (any line) in the cache.
    - **Mechanism:** The main memory address is divided into **Tag** and **Block Offset**. The entire tag must be compared against all tags in the cache simultaneously to find a match.
    - **Advantages:** Very low conflict misses, high hit rate for a given cache size.
    - **Disadvantages:** Complex and expensive hardware (requires many comparators); slower lookup due to parallel comparisons; generally used for very small caches (e.g., TLBs - Translation Lookaside Buffers).
3. **Set-Associative Cache:**
    - **Compromise:** Combines aspects of both direct mapped and fully associative.
    - **Mechanism:** The cache is divided into "sets." Each main memory block maps to a specific "set" (like direct mapped), but within that set, it can be placed in *any* of the available "ways" (like fully associative).
    - **Address:** Divided into **Tag, Set Index, Block Offset.**
        - **Set Index:** Determines which set the block maps to.
        - **Tag:** Used to identify the specific block within that set.
    - **Common Configurations:** 2-way, 4-way, 8-way set-associative.
    - **Advantages:** Better hit rate than direct mapped (fewer conflict misses) and simpler hardware than fully associative. This is the most common mapping technique in modern CPUs.

3.3. Cache Replacement Policies (Brief)

When a cache miss occurs and the target cache line is full, a **replacement policy** determines which existing block in the cache should be evicted to make space for the new block.

- **LRU (Least Recently Used):**
    - Evicts the block that has not been accessed for the longest time.
    - **Assumption:** What was used least recently is least likely to be used again soon (based on temporal locality).

    - **Implementation:** Complex to implement accurately, especially for large caches (requires tracking access times for all blocks). Approximations are often used.

- **FIFO (First-In, First-Out):**
    - Evicts the block that has been in the cache for the longest time.
    - **Assumption:** Blocks that have been in the cache longest are less likely to be needed.
    - **Implementation:** Simple (requires a queue).
    - **Disadvantage:** Can evict frequently used blocks if they were loaded early.
- **Random:**
    - Randomly selects a block to evict.
    - **Advantages:** Very simple to implement.
    - **Disadvantages:** Can lead to poor performance in some cases, as it doesn't consider usage patterns.

3.4. Write Policies

When data is written to a location that is present in the cache, the cache and main memory might become inconsistent. Write policies determine when the changes in the cache are reflected in main memory.

- **Write-Through:**
    - When data is written to the cache, it is **immediately also written to main memory.**
    - **Advantages:** Main memory is always consistent with the cache. Simpler to implement.
    - **Disadvantages:** Every write operation incurs a main memory access, which is slow, reducing write performance.
- **Write-Back (Copy-Back):**
    - When data is written to the cache, it is *only* updated in the cache initially. A "dirty bit" is set for that cache line.
    - The updated (dirty) block is written back to main memory **only when it is evicted from the cache.**
    - **Advantages:** Much better write performance as main memory writes are minimized (only occur on eviction or explicit flush).

- **Disadvantages:** More complex to implement (requires dirty bits). Main memory can be inconsistent with the cache for short periods (requires more complex cache coherency protocols in multi-processor systems).
- **Most Common:** Modern CPUs primarily use write-back caches for performance.

Review Questions:

1. Explain how the memory hierarchy improves overall system performance. In your answer, describe the trade-offs between speed, cost, and capacity across different memory levels.
2. Differentiate between **cache memory** and **main memory** in terms of:
   a) Access time
   b) Volatility
   c) Cost per bit
   d) Typical size
3. A CPU experiences frequent delays when accessing data from main memory. Suggest and explain two ways in which cache memory can help reduce these delays.
4. A CPU makes 10 memory requests. Out of these, 7 requests are found in the cache.
   a) Calculate the **cache hit ratio**.
   b) Calculate the **cache miss ratio**.

Hint:

$$\text{Hit Ratio} = \frac{\text{Number of Hits}}{\text{Total Accesses}}, \text{Miss Ratio} = 1 - \text{Hit Ratio}$$

5. Consider a **Direct Mapped Cache** with **4 cache lines**. Each memory block address maps to a cache line using:

$$\text{Cache Line} = (\text{Block Address}) \bmod 4$$

The CPU requests memory blocks in this sequence: `2, 6, 10, 2, 14, 6, 2, 18`

a) Identify for each request whether it results in a **hit** or **miss**.
b) Compute the **total number of hits and misses**.
c) Calculate the **cache hit ratio**.